

(2.) **Critical:** Generate a fine-grained personal access token (PAT) for remote authentication:

- Go to <https://github.com/settings/personal-access-tokens/> and select “Generate new token”.
- Provide a token name and expiration date.
- Choose “All repositories” to allow access to future repositories, or explicitly select existing ones.

Hint: To get a token for a collaboration project, the repository owner must have first added you as a collaborator.

- Under “Repository permissions”, set “Contents” to “Read and write”.
- Click “Generate token” and copy the generated string to your clipboard.

Hint: After the first clone or push, Git may prompt for your GitHub credentials. Use your username and paste your fine-grained PAT as the password; Git is typically configured to securely store your token using the system password manager via `git config --global credential.helper manager` (Windows) or `osxkeychain` (macOS).

(3.) Create a copy of the remote repository on your local machine, or link a local repository to the remote, by executing one of the following commands:

```
git clone gh-repo-url.git
git remote add origin gh-repo-url.git
```

(4.) To retrieve changes from the remote repository, execute:

```
git fetch origin
git merge origin/main
```

Hint: This two-step method is safer for avoiding unintended overwrites or merge conflicts when local file changes exist. `git pull origin main` is faster but may create automatic merge commits or conflicts if your local branch has diverged.

(5.) Upload your committed local changes to the remote repository with

```
git push origin main
```

» **Collaborating on remote repositories with GitHub**

(1.) As repository owner, add collaborators to your repository; set privileges as appropriate.

Hint: For public repositories, users can also contribute via pull requests without being added explicitly.

(2.) **Optional:** As repository owner, create a development branch for the project once

```
git checkout -b dev
git push origin dev
```

(3.) As a collaborator, accept the invitation, pull the repository, and create a new branch for your edits to isolate changes and make reviewing easier.

```
git pull origin dev # or "main" if no development branch exists
git checkout -b my-feature
git push origin my-feature
```

(4.) Edit, stage, commit, and push changes on your branch as usual.

This is why: Pushing will make your branch visible to other collaborators and allows you to create a pull request (PR).

Critical: Pull the latest changes regularly from the remote repository to keep your local repository up to date.

(5.) Once your branch is ready to be merged, create a pull request on GitHub:

- Navigate to the repository on GitHub, click “Compare & pull request”.

Hint: For production repositories, request a merge from the feature branch into the `dev` branch for active development first. Merge the `dev` branch to the `main` branch for stable releases.

- Respond to comments and update the pull request as needed.

Note: You can push additional commits to the same branch to update an open pull request.

- Once approved, the pull request will be merged into the development or the main branch.

🔗 []

» Linking repositories with submodules

- (1.) Use a submodule when part of one repository (private content, shared utilities, a vendored dependency) needs its own version history but should appear as a subdirectory of the parent. The parent pins a specific commit of the submodule, so updates are explicit and reproducible.

This is why: A submodule is one repository nested inside another. The parent stores the submodule's commit SHA, not its files. Pulling the parent never silently updates the submodule.

- (2.) Add a submodule to the parent repository.

```
git submodule add gh-other-repo-url.git submodule/path/in/parent
git commit -m "Add submodule/path/in/parent as submodule"
```

- (3.) When others clone the parent, initialize submodules in the same step or after the fact:

```
git clone --recurse-submodules gh-parent-repo-url.git
git submodule update --init --recursive
```

Note: The second form initializes submodules in an existing clone that was made without `--recurse-submodules`.

- (4.) To update the submodule to a newer commit, update the pointer in the parent.

```
cd submodule/path/in/parent
git pull origin main
cd ..
git add submodule/path/in/parent
git commit -m "Bump submodule pointer"
```

Critical: Editing files inside the submodule does *not* automatically update the parent's pointer. You must commit twice: once inside the submodule, once in the parent.

- (5.) *Optional:* To remove a submodule:

```
git rm path/in/parent
git commit -m "Remove submodule path/in/parent"
```

Hint: This removes the entry from `.gitmodules` and the working tree. The submodule's own repository is untouched.

» Preserving file history when renaming or splitting repositories

- (1.) For renames *within* a single repository, use `git mv` rather than a plain shell `mv` followed by `git add`.

Hint: Git stores content, not paths. `git mv` stages a rename automatically; a plain `mv` registers as a delete + add, which Git can still detect as a rename if content similarity is high enough, but the intent is less explicit.

- (2.) Confirm Git detected the rename.

```
git status          # "renamed: old -> new"
git log --follow new # history across the rename
```

This is why: By default, `git log` stops at the rename. The `--follow` flag traces history through it.

- (3.) To move files *between* repositories while preserving history, `git mv` is not enough — it only operates within one repository. Use `git filter-repo` to extract the relevant history into a portable form, then merge it into the target repository.

Resource: Workflows are more involved, consult <https://github.com/newren/git-filter-repo>.



T > Repository transfer

- (1.) Audit the repository.
- No pending branches, unmerged pull requests, or unpublished changes?
 - No dependencies on private data or credentials?
- (2.) Finalize README.md to reflect the latest project title, description, and status of the project.
- (3.) Create a final release tag so there is a citable, frozen snapshot of the code at time of handoff.

```
git tag v1.0.0-final
git push origin v1.0.0-final
```

- (4.) *Optional:* For published work, link the repository to Zenodo.

Resource: The repository is then stored safely for the future in CERN's Data Centre (<https://zenodo.org/>) for as long as CERN exists. Every upload is assigned a digital object identifier (DOI), making it citable and trackable.



- (5.) *Critical:* If the repository is associated with a personal GitHub account, transfer the repository to a designated maintainer, the lab's GitHub account or the institutional GitHub organization. Ensure a maintainer is assigned for substantial software projects.



Note: The target organization must be configured to accept outside transfers.

- (6.) Remove the departing member's write/admin access after transfer is complete.
- (7.) Archive the repository if it is no longer maintained. Archived repositories are read-only.
- (8.) Add the repository title, archive status, and DOI to the lab's knowledgebase or data inventory.

List of references

Change log

2025-07-26 Benjamin C. Buchmuller Initial commit.
2025-10-25 Benjamin C. Buchmuller Clarified branch/merge workflow and revised code.
2025-10-31 Benjamin C. Buchmuller Clarified credential manager setup.
2026-05-12 Benjamin C. Buchmuller Add procedures for submodules and rename history preservation.

From the *Lab Protocols* collection by B. C. Buchmuller and contributors (2026). Licensed under Creative Commons Attribution Share Alike 4.0 International; see <https://creativecommons.org/licenses/by-sa/4.0/> for the terms.

For research use. Provided in good faith, without warranty or liability for any use or results. Users are responsible for compliance with local regulations and institutional policies.

Current when printed. Visit <https://benjbuch.github.io/check/> or scan the QR code to check for updates.



94670e8

